Capstone Project

OpenCart - Capstone Project

QA automation capstone scope document using Playwright with JavaScript.

Field	Details
Project type	E-commerce website functional testing capstone
Selected website	OpenCart Demo Store
Website URL	https://naveenautomationlabs.com/opencart/
Automation stack	Playwright with JavaScript
Minimum requirement	8 services, minimum of 80 test cases
Testing focus	Frontend user flows, form validations, cart and checkout behavior, account-related pages, and reporting evidence

Project Overview

The primary objective of this capstone is to design and execute a comprehensive QA automation suite for the OpenCart demo application. This project focuses strictly on quality assurance methodologies and test execution.

OpenCart serves as an ideal environment for this testing framework because it fully simulates a live, data-driven e-commerce platform. By validating essential consumer journeys, such as account registration, product searches, cart manipulation, and the end-to-end checkout pipeline, the project will effectively demonstrate real-world testing capabilities and ensure functional stability across the application, the site is suitable for a capstone project that requires at least 80 total test cases.

Project Objectives

- Prepare a complete test plan for the selected website.
- Create a minimum of 80 test cases across 8 services.
- Cover positive, negative, validation, navigation, and regression scenarios.
- Automate the planned tests using Playwright with JavaScript.
- Generate execution evidence through test reports, screenshots, and traces wherever required.
- Present the project as a QA/testing capstone, not as a development project.

Scope Of Testing

The scope is limited to the public frontend demo store. The project will focus on customer-facing functionality and avoid any activity that requires real payment, real personal information, or backend administration.

In Scope	Out Of Scope
Authentication: Registration, login/logout, and account navigation.	**Financials:** Real payment processing and actual order fulfillment.
Catalog: Product searches, category browsing, sorting, and detail pages.	**Backend:** Database validations and OpenCart admin panel modifications.
Transactions: Cart manipulation, product comparisons, and checkout UI flows.	**Non-Functional:** Performance, security, stress, or load testing.
Fulfillment: Address book updates, shipping steps, and order history logs.	**Real Data:** Testing with actual customer PII or real credit card details.
Support & API: Contact forms, footer links, and internal API validations.	**Environment:** Altering server configurations, product pricing, or core code.

Automation Approach

The automation framework will be built using Playwright with JavaScript. The suite will be organized strictly by service to guarantee that each OpenCart module can be maintained and executed independently.

- **Core Runner:** Utilize Playwright Test as the primary execution engine for the suite.
- **Modular Specs:** Construct separate .spec.js files for each major OpenCart service, such as authentication, cart, checkout, and user profile.
- **Resilient Locators:** Prioritize stable, user-centric locators like roles, labels, text placeholders, and visible UI elements to minimize script flakiness.
- **Strict Assertions:** Implement hard assertions to constantly verify page URLs, success messages, product titles, cart totals, and final order confirmations.
- **Reusable Setup:** Leverage Playwright hooks and helper functions to handle repetitive prerequisites, like user logins or adding baseline products to the cart.
- **Data Independence:** Keep test data entirely separate from automation logic, allowing dummy names, emails, and addresses to be swapped out seamlessly.

Service-Wise Test Case Distribution

The capstone requirement will be met by preparing 10 test cases for each of the following 8 services. This gives a planned total of 80 test cases irrespective of browser selection.

No.	Service / Module	Testing coverage	Count
1	Authentic ation	Register, login, invalid credentials, logout, and password recovery.	15
2	Product Management	Search functionality, category navigation, sorting, product details, and reviews.	15
3	Shopping Cart	Add to cart, update quantity, remove item, subtotal calculation, and mini-cart.	15
4	Checkout and Payment	Guest checkout, registered checkout, billing/shipping steps, and order confirmation.	15
5	User Profile	Edit account info, change password, order history, and newsletter subscription.	15
6	Address and Shipping	Add/edit/delete address, default address behavior, and address validation.	11
7	Customer Support	Contact Us form, product returns, site map, and footer link navigation.	11
8	Internal API	API status checks, login payloads, cart manipulation via API, and response times.	05

Planned Project Structure

```
Playwright-Capstone-Project/
|--- .auth/
|      |--- user.json
|--- .github/
|      |--- workflows
|           |--- playwright.yml
|--- allure-report/
|
|--- allure-results/
|
|--- .gitignore
|--- package.json
|--- playwright.config.js
|--- package-lock.json
|
|--- tests/
|      |--- authentication.spec.js
|      |--- productManagement.spec.js
|      |--- shoppingCart.spec.js
|      |--- checkoutPayment.spec.js
|      |--- userProfile.spec.js
|      |--- addressShipping.spec.js
|      |--- customerSupport.spec.js
|      |--- internalApi.spec.js
|
|--- pages/
|      |--- BasePage.js
|      |--- LoginPage.js
|      |--- RegisterPage.js
|      |--- ForgotPasswordPage.js
|      |--- ProductPage.js
|      |--- CartPage.js
|      |--- CheckoutPage.js
|      |--- UserProfilePage.js
|      |--- AddressShippingPage.js
|      |--- CustomerSupportPage.js
|
|--- utils /
|      |--- global-setup.js
|
|---playwright-report/
```

Test Data Strategy

The OpenCart platform is a public demo environment, all test execution will rely exclusively on dummy data to ensure safe, repeatable test runs.

- **Synthetic User Info:** Generate unique dummy emails, names, and phone numbers dynamically to prevent duplicate account registration errors during auth testing.
- **Zero Real PII:** Strictly avoid the use of actual customer data, personal information, or real credit card details during the checkout and payment flows.
- **Existing Catalog Utilization:** Leverage the native products already populated in the OpenCart demo rather than attempting to inject new items into the database.
- **Reusable Data Objects:** Centralize test data payloads—like standard billing addresses, shipping details, and contact form inputs within the test-data/ directory for easy maintenance.
- **Environment Reset Handling:** Design the automation scripts to be resilient against periodic database resets by creating fresh user accounts and carts at the start of required test runs.

Entry And Exit Criteria

Entry criteria	Exit criteria
The OpenCart demo store is live and major pages are accessible.	All 80 test cases are fully documented across the 8 chosen services.
The selected 8 testing modules and target scenarios are finalized.	Playwright automation scripts are written and execute successfully for the planned scope.
The Playwright + JavaScript environment and POM structure are set up.	The final Playwright HTML execution report is generated and reviewed.
Disposable dummy test data objects are prepared.	Any failing OpenCart flows are properly recorded with evidence.
The browser execution approach (e.g., Chromium, Firefox) is configured.	Final capstone files are organized for submission.

Risks And Mitigation

Risk	Mitigation
Demo site resets regularly	Use fresh dummy data and avoid depending on old registered accounts.
Product names or prices may change	Use flexible assertions and update test data if visible website data changes.
Public site may be slow or temporarily unavailable	Use Playwright timeouts carefully and rerun after confirming site availability.
Some checkout paths may need specific product types	Select products that support predictable cart and checkout behavior.

Automation locators may break if UI changes	Prefer stable user-facing locators and keep selectors organized.

Assumptions

- The capstone will run against the public frontend demo store at https://naveenautomationlabs.com/opencart/
- Only dummy test data will be used.
- The primary automation framework will be Playwright with JavaScript.
- The 80 test cases will be counted service-wise, not browser-wise.
- If the website resets or changes, affected test cases will be updated with trainer approval.

Planned Capstone Flow

1. Finalize website and service-wise testing scope.
2. Prepare minimum of 80 test cases across 8 services.
3. Review and improve test cases based on trainer feedback.
4. Create Playwright with JavaScript project structure.
5. Automate tests service by service.
6. Execute tests, review failures, and fix automation issues.
7. Prepare final report, screenshots/traces, and submission folder.

Test Execution Commands

Run All Tests

npx playwright test

Run Tests in Headed Mode

npx playwright test --headed

Run Tests on Chromium Browser

npx playwright test --project "chromium"

Open Report

npx playwright show-report

Conclusion

The selected website and planned scope are suitable for the capstone requirement because the application has enough independent services to prepare minimum of 80 test cases and enough realistic e- commerce behavior to demonstrate QA automation skills. The project will be completed using Playwright with JavaScript, matching the training stack and keeping the work focused on testing activities.